

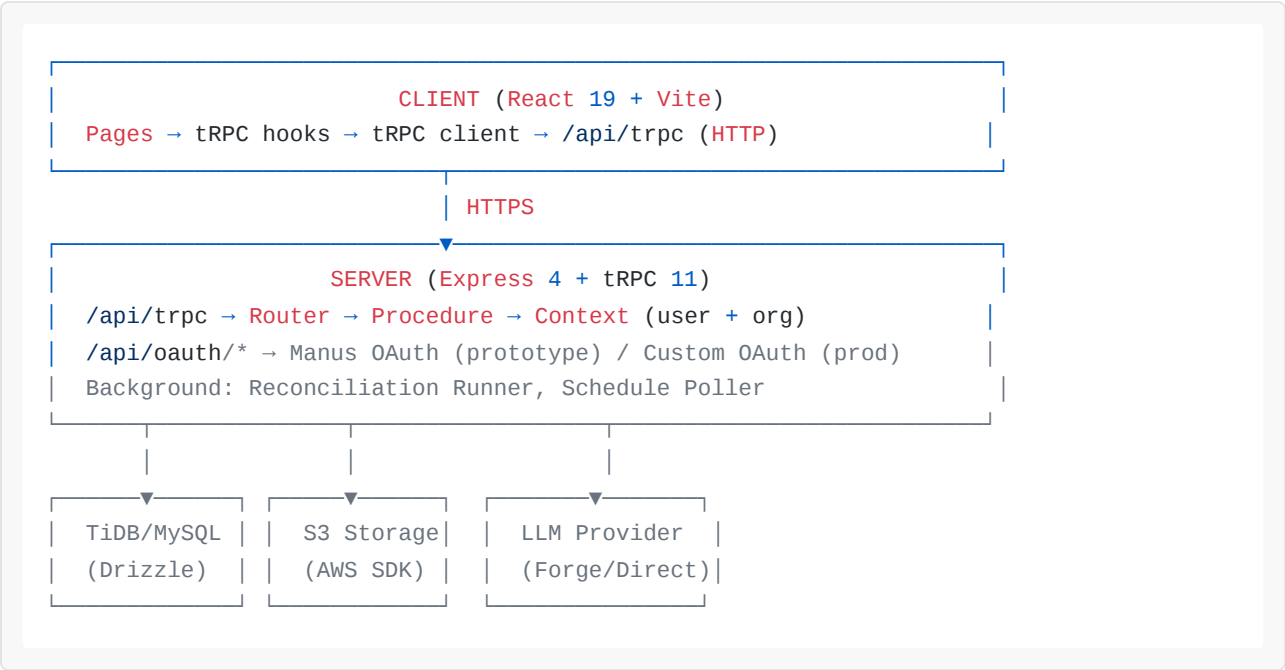
ReconcileAI — Architecture Document

Version: 2.0 — Lapo MFB Pilot Edition
Date: May 2026
Author: Richard Anwanakak, Founder & CPO — Infinity AI Africa Limited
Status: Active — Handoff to Rocket.new

1. Architecture Overview

ReconcileAI is a **multi-tenant SaaS platform** built on a monorepo full-stack architecture. The prototype was built on the Manus WebDev platform (Node.js, React, TiDB). The production build will be deployed on Rocket.new and served at <https://reconcileai.vip/>.

1.1 High-Level Architecture



1.2 Three-Layer Product Architecture

ReconcileAI’s reconciliation logic is organised into three conceptual layers:

Layer 1: Balance Engine

- Multi-source ingestion (CSV, API, SFTP)
- 5-pass matching algorithm
- Duplicate detection
- Reversal tracking

Layer 2: Exception Classifier

- Category classification (8 categories)
- Severity scoring (low/medium/high/critical)
- AI analysis and suggested resolution
- Assignment and escalation workflow

Layer 3: Context-Aware Agent (Super Agent)

- Natural language Q&A over reconciliation data
 - Action draft layer (propose → human approve → execute)
 - Semantic memory (org-specific pattern learning)
-

2. Technology Stack

2.1 Core Stack

Layer	Technology	Version	Notes
Frontend framework	React	19	With React Query via tRPC
Build tool	Vite	6	HMR in dev, optimised bundle in prod
Styling	Tailwind CSS	4	OKLCH colour format in @theme blocks
UI components	shadcn/ui + Radix UI	Latest	Full component library
Routing	Wouter	3	Lightweight client-side router
Backend framework	Express	4	Single Node.js process
API layer	tRPC	11	End-to-end type safety, no REST
Serialisation	Superjson	2	Dates, BigInt, Maps preserved
ORM	Drizzle ORM	0.41	Schema-first, type-safe queries
Database	TiDB (MySQL-compatible)	Cloud	Distributed, HA, MySQL wire protocol
File storage	AWS S3	SDK v3	All file bytes; DB stores metadata only
Auth	JWT (jose)	5	httpOnly session cookies
Validation	Zod	3	Input validation on all procedures
Testing	Vitest	3	Unit tests in <code>server/*.test.ts</code>
Language	TypeScript	5	Strict mode throughout

2.2 Key Libraries

Library	Purpose
<code>exceljs</code>	Excel report generation (XLSX)
<code>jspdf + html2canvas</code>	PDF report generation
<code>papaparse</code>	CSV parsing (client-side)
<code>ssh2-sftp-client</code>	SFTP ingestion
<code>recharts</code>	Data visualisation (charts)
<code>framer-motion</code>	UI animations
<code>streamdown</code>	Streaming markdown rendering (Super Agent)
<code>react-hook-form + zod</code>	Form validation
<code>date-fns</code>	Date manipulation
<code>nanoid</code>	Unique ID generation
<code>sonner</code>	Toast notifications

3. Repository Structure

```
reconcileai/
├─ client/
│  ├─ public/           ← Static assets (served at /)
│  └─ src/
│     ├─ pages/         ← 50+ page-level components
│     ├─ components/    ← Reusable UI (DashboardLayout, AIChatBox, Map, etc.)
│     ├─ contexts/      ← React contexts (PortalContext, AuthContext)
│     ├─ hooks/         ← Custom hooks
│     ├─ lib/trpc.ts     ← tRPC client binding
│     ├─ App.tsx        ← Routes and layout
│     ├─ main.tsx       ← Providers tree
│     └─ index.css      ← Global styles and CSS variables
├─ server/
│  ├─ _core/            ← Framework plumbing (DO NOT EDIT)
│  │  ├─ context.ts     ← tRPC context (user, org, role)
│  │  ├─ trpc.ts        ← publicProcedure, protectedProcedure, router
│  │  ├─ llm.ts         ← LLM provider abstraction (Forge / Direct)
│  │  ├─ env.ts         ← Environment variable registry
│  │  ├─ notification.ts ← Owner notification helper
│  │  ├─ imageGeneration.ts ← Image generation helper
│  │  └─ voiceTranscription.ts ← Whisper transcription helper
│  ├─ routers/
│  │  └─ cbnCompliance.ts ← CBN compliance sub-router
│  ├─ routers.ts        ← Main tRPC router (5500+ lines, 40+ sub-routers)
│  ├─ db.ts             ← Query helpers
│  └─ storage.ts        ← S3 helpers (storagePut, storageGet)
├─ drizzle/
│  ├─ schema.ts         ← All table definitions (1450+ lines, 50+ tables)
│  └─ migrations/       ← Auto-generated migration files
├─ docs/               ← Handoff documentation (this folder)
├─ shared/             ← Shared constants and types
├─ package.json
├─ .env.example        ← Sanitised environment variable template
└─ README.md
```

4. Database Schema

4.1 Core Tables

Table	Purpose	Key Fields
users	Platform users	id, email, name, role, organizationId
organizations	Tenants	id, name, segment (financial_services/corporate_b2b/super_admin)
channels	Data sources per org	id, organizationId, channelType, fieldMapping
upload_batches	File upload tracking	id, channelId, fileHash, status, totalRows
transactions	All ingested transactions	id, batchId, channelId, amount, transactionRef, status
reconciliation_jobs	Reconciliation runs	id, sourceChannelId, targetChannelId, moduleType, status, matchRate
matches	Confirmed transaction pairs	id, jobId, sourceTransactionId, targetTransactionId, matchType, confidenceScore
exceptions	Unmatched/problematic transactions	id, jobId, category, severity, status, aiAnalysis
audit_logs	Immutable action log (org-level)	id, userId, action, entityType, details
platform_audit_logs	Immutable action log (platform-level)	id, actorId, targetOrgId, action, details

4.2 Supporting Tables

Table	Purpose
<code>scheduled_tasks</code>	Scheduled reconciliation jobs
<code>schedule_run_history</code>	Run history per scheduled task
<code>reconciliation_reports</code>	Generated report metadata + S3 URL
<code>shared_report_tokens</code>	Time-limited public report access tokens
<code>webhooks</code>	Outbound webhook configurations
<code>api_keys</code>	External API key management (hashed)
<code>sftp_credentials</code>	SFTP connection credentials (encrypted)
<code>sftp_ingestion_logs</code>	SFTP pull history
<code>api_ingestion_logs</code>	API pull history
<code>email_preferences</code>	Email notification settings per org
<code>user_role_preferences</code>	Per-user UI preferences (view, theme, digest)
<code>anomaly_scores</code>	Detected anomalies with scores
<code>detection_rules</code>	Configurable anomaly detection rules
<code>resolution_templates</code>	Reusable exception resolution templates
<code>module_configurations</code>	Per-org module enable/disable state
<code>module_overrides</code>	Super admin per-institution module overrides
<code>job_progress_events</code>	Real-time job progress events
<code>dashboard_stats_cache</code>	Cached dashboard metrics (TTL-based)

4.3 Compliance Tables

Table	Purpose
<code>compliance_settings</code>	NDPA/NDPR org-level settings
<code>data_deletion_requests</code>	NDPA Article 7 deletion requests
<code>security_incidents</code>	NDPA breach notification log
<code>compliance_assessments</code>	Public self-assessment tool results
<code>cbnReportFrameworks</code>	CBN report template definitions
<code>cbnReportSubmissions</code>	CBN submission tracking
<code>cbnReportFindings</code>	Findings per submission
<code>cbnActionPlans</code>	Action plans per finding
<code>cbnAuditLog</code>	CBN module audit trail
<code>cbnDeadlineSubmissions</code>	Deadline tracking

4.4 Agent Tables

Table	Purpose
<code>agent_action_drafts</code>	Super Agent proposed actions awaiting approval
<code>agent_memory</code>	Semantic memory entries per org
<code>guest_sessions</code>	Guest demo session state
<code>guest_tokens</code>	Guest access tokens

4.5 Multi-Tenancy Enforcement

Every business data table includes `organizationId`. All tRPC procedures that access business data scope queries to `ctx.user.organizationId`. The `superAdminProcedure` middleware is the only path that can access cross-tenant data, and only for users with `role = super_admin`.

5. API Layer (tRPC Routers)

All client-server communication goes through tRPC at `/api/trpc`. There are no separate REST endpoints for business logic.

5.1 Router Map

Router	Key Procedures
auth	me , logout , updateProfile
channels	list , create , update , delete , getAlertSettings
upload	createBatch , processFile , getBatchStatus
transactions	list , getById , updateStatus
reconciliation	createJob , runJob , getJob , listJobs , cancelJob
exceptions	list , getById , assign , resolve , escalate , dismiss , bulkUpdate
resolutionTemplates	list , create , update , delete
modules	getConfig , updateConfig
review	listPendingMatches , confirmMatch , rejectMatch
audit	list , export
reports	list , create , getById , share , exportPdf , exportExcel
export	exceptions , transactions , auditTrail
dashboard	getStats , getJobSummary , getExceptionSummary
sampleData	generate
webhooks	list , create , update , delete , test
apiKeys	list , create , revoke
sftp	list , create , update , delete , test , triggerPull
schedules	list , create , update , delete , runNow
emailPreferences	get , update
monitoring	getJobProgress , getSystemHealth
anomalies	list , getById , updateStatus
detectionRules	list , create , update , delete
admin	listUsers , inviteUser , updateUserRole , deactivateUser

Router	Key Procedures
superAdmin	getStats , listOrgs , createOrg , listUsers , getOrgContext , toggleOrgModule , listOrgOverrides , setOrgModuleOverride , clearOrgModuleOverride
superAgent	chat , listDrafts , approveDraft , rejectDraft , getMemory
compliance	getSettings , updateSettings , listDeletionRequests , listIncidents
assessment	start , submitAnswer , getResult
woodcore	getHealth , getAccounts , getTransactions (POC — pending IP whitelist)
cbnCompliance	listFrameworks , createSubmission , addFinding , createActionPlan
cfoReports	getSchedule , createSchedule , updateSchedule , sendNow
leads	create (public — demo request form)
docs	list , getById (documentation viewer)

5.2 Procedure Middleware Chain

```

publicProcedure
  └─ No auth required (login page, public landing, demo request)

protectedProcedure
  └─ Requires valid JWT session cookie
  └─ ctx.user = { id, email, name, role, organizationId }

adminProcedure (extends protectedProcedure)
  └─ Requires role = 'admin' OR role = 'super_admin'

superAdminProcedure (extends protectedProcedure)
  └─ Requires role = 'super_admin' AND organizationId = Infinity AI org ID
  └─ Grants cross-tenant data access

```

6. Authentication

6.1 Prototype (Manus OAuth)

The prototype uses Manus OAuth 2.0. The flow is:

1. Frontend calls `getLoginUrl()` → redirects to Manus login portal

2. Manus redirects to `/api/oauth/callback?code=...&state=...`
3. Server exchanges code for user info, creates/updates user record, issues JWT session cookie
4. All subsequent requests carry the cookie; `server/_core/context.ts` validates it and populates `ctx.user`

6.2 Production (Rocket.new)

When moving to Rocket.new, replace Manus OAuth with one of:

Option A — Email/password + magic link (recommended for MFB pilot)

- Use a library like `better-auth` or `lucia-auth`
- Add `password_hash` column to `users` table
- Implement `/api/auth/login`, `/api/auth/magic-link`, `/api/auth/verify` endpoints
- Keep the JWT session cookie pattern (already in place)

Option B — Clerk or Auth0

- Drop-in authentication service
- Replace `server/_core/context.ts` JWT validation with Clerk/Auth0 SDK
- Map external user ID to internal `users` table on first login

Option C — Keep Manus OAuth (if staying on Manus infrastructure)

- No changes required

Critical: The `VITE_OAUTH_PORTAL_URL`, `OAUTH_SERVER_URL`, and `VITE_APP_ID` environment variables are Manus-specific. These must be replaced with the new auth provider's equivalents.

7. LLM Integration (Detailed)

This section is critical for the Rocket.new handoff. The Manus Forge gateway will not be available in production.

7.1 Current Implementation

The `invokeLLM()` function in `server/_core/llm.ts` is the single entry point for all LLM calls. It resolves the provider at runtime based on environment variables:

```
// Provider resolution logic (simplified)
function resolveProvider(): ProviderConfig {
  const directKey = process.env.DIRECT_LLM_API_KEY;

  if (directKey && directKey.trim() !== "") {
    // Mode 2: Direct provider (OpenAI, Anthropic, etc.)
    return {
      mode: "direct",
      apiUrl: process.env.DIRECT_LLM_API_URL ??
"https://api.openai.com/v1/chat/completions",
      apiKey: directKey,
      model: process.env.DIRECT_LLM_MODEL ?? "gpt-4o",
    };
  }

  // Mode 1: Manus Forge (prototype only)
  return {
    mode: "forge",
    apiUrl: process.env.BUILT_IN_FORGE_API_URL ?? "https://forge.manus.im",
    apiKey: process.env.BUILT_IN_FORGE_API_KEY,
    model: "gemini-2.5-flash",
  };
}
```

7.2 Where LLM is Called in the Codebase

Search for `invokeLLM` in `server/routers.ts` to find all call sites. Key locations:

Router	Procedure	LLM Purpose
<code>exceptions</code>	<code>getById</code> , <code>list</code>	Generate <code>aiAnalysis</code> and <code>suggestedResolution</code> per exception
<code>reconciliation</code>	<code>runJob</code> (background runner)	AI-suggested match scoring for ambiguous pairs
<code>superAgent</code>	<code>chat</code>	Natural language Q&A
<code>superAgent</code>	<code>approveDraft</code>	Action execution confirmation
<code>anomalies</code>	<code>getById</code>	Anomaly narrative generation

7.3 Production Setup

Step 1 — Add environment variables to Rocket.new:

```
DIRECT_LLM_API_KEY=sk-...           # OpenAI API key
DIRECT_LLM_API_URL=https://api.openai.com/v1/chat/completions
DIRECT_LLM_MODEL=gpt-4o
```

No code changes required. The `invokeLLM()` helper will automatically switch to the direct provider.

Step 2 — For Anthropic instead of OpenAI:

```
DIRECT_LLM_API_KEY=sk-ant-...
DIRECT_LLM_API_URL=https://api.anthropic.com/v1/messages
DIRECT_LLM_MODEL=claude-3-5-sonnet-20241022
```

Note: Anthropic's API format differs slightly from OpenAI. The current `invokeLLM()` implementation uses the OpenAI-compatible format. If using Anthropic directly, either use an OpenAI-compatible proxy (e.g., LiteLLM) or update `server/_core/llm.ts` to handle Anthropic's message format.

Step 3 — Recommended: Use LiteLLM as a unified proxy

LiteLLM provides a single OpenAI-compatible endpoint that routes to any provider:

```
# Run LiteLLM proxy
docker run -p 4000:4000 ghcr.io/berriai/litellm:main \
  --model gpt-4o --api_key $OPENAI_API_KEY

# Then set:
DIRECT_LLM_API_KEY=sk-...
DIRECT_LLM_API_URL=http://localhost:4000/v1/chat/completions
DIRECT_LLM_MODEL=gpt-4o
```

7.4 Streaming (Super Agent)

The Super Agent's chat interface uses `streamdown` for streaming markdown rendering. The current `invokeLLM()` returns a complete response. To enable streaming in production:

1. Add a `stream: true` parameter to `InvokeParams` in `server/_core/llm.ts`
2. When `stream: true`, use `fetch()` with `response.body` as a `ReadableStream`
3. Expose the stream via a tRPC subscription or a dedicated SSE endpoint at `/api/stream/agent`
4. On the client, consume via `EventSource` or the tRPC subscription hook

7.5 Cost Estimation (Lapo MFB Pilot)

Assuming 50,000 transactions/day and 5% exception rate (2,500 exceptions/day):

Use Case	Calls/day	Tokens/call	Cost/day (GPT-4o-mini)
Exception analysis	2,500	500	\$0.75
AI-suggested matching	500	300	\$0.09
Super Agent queries	50	1,000	\$0.03
Total			\$0.87/day

At scale (500,000 tx/day): ~\$8.70/day. Build per-org usage tracking and include LLM cost in the SaaS pricing model.

8. File Storage

All file bytes are stored in AWS S3. The database stores only metadata (URL, key, size, MIME type).

8.1 S3 Helpers

```
// server/storage.ts
import { storagePut } from "../server/storage";

const { url, key } = await storagePut(
  `${orgId}/uploads/${batchId}-${fileName}`,
  fileBuffer,
  "text/csv"
);
// Store url and key in upload_batches table
```

8.2 File Lifecycle

1. Client uploads file → POST to `/api/trpc/upload.createBatch`
2. Server validates file, calls `storagePut()` → gets S3 URL
3. Server stores URL in `upload_batches.fileUrl`
4. Background reconciliation runner fetches file from S3 URL for processing
5. Processed files are retained for 90 days (configurable), then deleted via S3 lifecycle policy

8.3 Production S3 Configuration

In Rocket.new, configure:

```
AWS_ACCESS_KEY_ID=...
AWS_SECRET_ACCESS_KEY=...
AWS_REGION=af-south-1      # Cape Town region (lowest latency for Nigeria)
AWS_S3_BUCKET=reconcileai-prod
```

Or use Cloudflare R2 (S3-compatible, no egress fees):

```
AWS_ACCESS_KEY_ID=...      # R2 access key
AWS_SECRET_ACCESS_KEY=...  # R2 secret key
AWS_REGION=auto
AWS_S3_BUCKET=reconcileai-prod
AWS_S3_ENDPOINT=https://<account_id>.r2.cloudflarestorage.com
```

9. Background Processing

9.1 Reconciliation Runner

The reconciliation engine runs as a background process within the same Node.js process. It is triggered by:

- Direct call from `reconciliation.runJob` procedure
- Scheduled task poller (every 60 seconds, checks for due scheduled tasks)

Location: `server/routers.ts` — `runReconciliationJob()` function (line 5346)

Processing flow:

1. **Fetch** source transactions (**from** DB, **scoped to** sourceChannelId + dateRange)
2. **Fetch** target transactions (**from** DB, **scoped to** targetChannelId + dateRange)
3. Pass 1: Exact **match** (transactionRef + amount + date)
4. Pass 2: Fuzzy reference **match** (normalised string similarity)
5. Pass 3: Amount tolerance **match** (**within** tolerance, same **date window**)
6. Pass 4: **Date window match** (exact amount, **date within window**)
7. Pass 5: AI-suggested **match** (LLM scores remaining unmatched pairs)
8. Write **matches** to ``matches`` table
9. Write exceptions to ``exceptions`` table
10. **Update** job stats (matchRate, exceptionCount, etc.)
11. Emit job progress events (**for real-time** monitoring)
12. **Trigger** webhooks (if configured)

9.2 Production Scaling Consideration

The current single-process background runner is sufficient for the pilot (up to ~500,000 transactions per job). For production scale (millions of transactions), the runner should be extracted to a dedicated worker service:

- Use a job queue (BullMQ + Redis, or AWS SQS)
- Each reconciliation job becomes a queue item
- Worker processes consume the queue independently
- This also enables horizontal scaling of the reconciliation engine

9.3 Schedule Poller

A `setInterval` loop runs every 60 seconds and checks `scheduled_tasks` for jobs due to run. This is a simple polling approach suitable for the pilot. For production, replace with a proper cron scheduler (node-cron, Agenda, or a cloud scheduler like AWS EventBridge).

10. Domain and Deployment

10.1 Target Domain

The production platform must be deployed at: `https://reconcileai.vip/`

The domain `reconcileai.vip` is already registered and currently pointing to the Manus-hosted prototype.

10.2 DNS Configuration for Rocket.new

When Rocket.new builds and deploys the production application:

1. Rocket.new will provide a deployment URL (e.g., `reconcileai.rocket.app` or similar)
2. In the DNS provider for `reconcileai.vip`, add a CNAME record:

```
Type: CNAME
Name: @ (or www)
Value: <rocket.new deployment URL>
TTL: 300
```

3. If Rocket.new supports custom domains (most production platforms do), add `reconcileai.vip` as a custom domain in the Rocket.new project settings
4. Rocket.new will provision a TLS certificate automatically (Let's Encrypt or similar)
5. Update the `www` subdomain to also point to the same deployment:

```
Type: CNAME
Name: www
Value: <rocket.new deployment URL>
```

10.3 Environment Variables for Production

See `.env.example` in the repository root for the full list. Critical variables:

```
# Database
DATABASE_URL=mysql://user:pass@host:port/dbname

# Auth (replace Manus OAuth with production auth)
JWT_SECRET=<32+ character random string>

# LLM (replaces Manus Forge)
DIRECT_LLM_API_KEY=<openai or anthropic key>
DIRECT_LLM_API_URL=https://api.openai.com/v1/chat/completions
DIRECT_LLM_MODEL=gpt-4o

# S3 / R2
AWS_ACCESS_KEY_ID=...
AWS_SECRET_ACCESS_KEY=...
AWS_REGION=af-south-1
AWS_S3_BUCKET=reconcileai-prod

# App
NODE_ENV=production
PORT=3000
```

11. Security Architecture

11.1 Authentication and Session

- JWT tokens signed with `JWT_SECRET` (HS256)
- Stored as `httpOnly`, `secure`, `sameSite=strict` cookies
- Token expiry: 7 days (configurable)
- No refresh token in prototype; add refresh token rotation for production

11.2 API Key Security

- API keys are stored as SHA-256 hashes in the `api_keys` table
- The raw key is shown only once at creation time
- Key rotation: revoke + create new

11.3 SFTP Credential Security

- SFTP passwords/private keys are stored encrypted in `sftp_credentials`
- Encryption key must be set as `SFTP_ENCRYPTION_KEY` environment variable in production
- In the prototype, encryption is handled by the Manus platform

11.4 Data Isolation

- Every query is scoped to `ctx.user.organizationId`
- Cross-tenant access requires `superAdminProcedure` middleware
- No shared state between tenants in the application layer

11.5 Input Validation

- All tRPC procedure inputs are validated with Zod schemas
- File uploads are validated for MIME type and size before processing
- SQL injection is prevented by Drizzle ORM’s parameterised queries

12. Monitoring and Observability

12.1 Current (Prototype)

- Server logs to stdout (captured by Manus platform)
- Job progress events stored in `job_progress_events` table
- Dashboard stats cached in `dashboard_stats_cache` (TTL-based)

12.2 Production Recommendations

Concern	Recommended Tool
Application monitoring	Sentry (error tracking + performance)
Log aggregation	Datadog or Logtail
Uptime monitoring	Better Uptime or Checkly
Database monitoring	PlanetScale Insights or TiDB Cloud monitoring
LLM cost tracking	LangSmith or custom usage table

13. Known Technical Debt

The following items were accepted as prototype shortcuts and must be addressed before production:

Item	Risk	Recommended Fix
Single-process background runner	Blocks request handling during large jobs	Extract to BullMQ worker
No refresh token rotation	Session hijacking risk after 7 days	Add refresh token with rotation
SFTP encryption key not externalised	Credentials at risk if DB is compromised	Set <code>SFTP_ENCRYPTION_KEY</code> env var
<code>transaction_integrity</code> enum value retained in <code>reconciliation_jobs</code>	Legacy data confusion	Migrate existing rows to <code>settlement</code> , remove enum value
No rate limiting on public procedures	DoS risk on <code>leads.create</code> , <code>assessment.*</code>	Add express-rate-limit middleware
Dashboard stats cache invalidation is manual	Stale data risk	Add TTL-based auto-invalidation
Super Agent streaming not implemented	UX degradation for long responses	Implement SSE streaming (see Section 7.4)
No database connection pooling config	Connection exhaustion under load	Set <code>max: 10</code> in Drizzle MySQL2 config

14. Woodcore Integration (Pending)

The Woodcore POC integration is implemented in `server/routers.ts` under the `woodcore` router (line ~3927). It is currently blocked by IP whitelisting on Woodcore's side.

What is implemented:

- `woodcore.getHealth` — health check against Woodcore test tenant
- `woodcore.getAccounts` — fetch account list
- `woodcore.getTransactions` — fetch transactions for a given account and date range
- Live data display in the WoodcorePOC page (`/woodcore-poc`)

What is needed to complete:

1. Woodcore to whitelist the production server's IP address
2. Set `WOODCORE_API_URL`, `WOODCORE_CLIENT_ID`, `WOODCORE_CLIENT_SECRET` in production environment

3. Map Woodcore transaction fields to ReconcileAI's transaction schema (field mapping in channel config)
4. Create a Woodcore channel type in the channel management UI

For Lapo MFB: Lapo does not use Woodcore. Their core banking integration will require a separate connector, likely via SFTP file export or a direct REST API integration with their core banking vendor.